1. Methodology

This section describes how the data was cleaned, how features and targets were created, how the models were trained and evaluated, and how experiments were tracked.

1.1 Data source

- **Dataset:** `india_housing_prices.csv`
- **Size:** 250,000 rows and 23 columns in the original file
- **Columns used:** State, City, Locality, Property_Type, BHK, Size_in_SqFt, Price_in_Lakhs, Year_Built, Furnished_Status, Floor_No, Total_Floors, Age_of_Property, Nearby_Schools, Nearby_Hospitals, Public_Transport_Accessibility, Parking_Space, Security, Amenities, Facing, Owner_Type, Availability_Status
- **Environment:** Google Colab, Python 3.13, scikit-learn 1.6.1, pandas 2.2.3, NumPy 2.1.3, XGBoost 3.4.1
- **Random seed:** 42 (used for all splits and models)

1.2 Data cleaning

Step	Action	Result
Duplicates	Checked all columns for duplicate rows	0 duplicates found
Identifier	Dropped the `ID` column	Not used as a feature
Missing values	Checked Price, Size and City for nulls	0 nulls found
Invalid values	Removed rows with Price or Size ⩽ 0	0 rows removed
Outliers	Capped Price_in_Lakhs and Size_in_SqFt at the 1st and 99th percentile	Rows kept, extreme values limited
Type conversion	Converted numeric features with `to_numeric(errors="coerce")`	Safe handling of bad text values

After cleaning, the dataset has **250,000 rows and 22 columns**.

Capping was used instead of deleting rows so that no property records were lost. Price per sq ft was not capped separately. Section 2 (EDA) shows that about 7.9% of raw price-per-sq-ft values are high outliers, and they were kept in the data.

1.3 Feature engineering

- **Price_per_SqFt** = Price_in_Lakhs × 1,00,000 ÷ Size_in_SqFt. The Price_per_SqFt column in the original file is on a different scale, so it was recalculated.
- **Amenities_Count** = number of items in the comma-separated Amenities list.
- **Age_of_Property** was taken from the dataset (it matches Current Year − Year_Built).
- **Categorical encoding:** One-hot encoding for State, City, Property_Type, Furnished_Status, Public_Transport_Accessibility, Facing, Owner_Type, Availability_Status, Security and Parking_Space. Parking_Space and Security contain Yes/No values, so they are treated as categories and not as numbers.
- **Scaling:** StandardScaler is applied to numeric features for Logistic Regression and Linear Regression only. Tree-based models do not need scaling.

The brief also suggested a School Density Score. It was not created, because the dataset has only a count of nearby schools, so a density measure could not be calculated without area data.

1.4 Target variables

Classification target: Good_Investment

A property is labelled **1 (Good Investment)** if its price per sq ft is below the median price per sq ft of its own city. Otherwise it is labelled **0**.

- This is a domain rule chosen from the options in the brief (price vs median, price per sq ft vs median, multi-factor score).
- Class balance: 125,013 properties labelled 0 and 124,987 labelled 1 (about 50% each).

Regression target: Future_Price_5yr_Lakhs

Future price is calculated with the compound growth formula:

> Future_Price = Price_in_Lakhs × (1 + r)^5

The growth rate `r` depends on the city:

City	Annual growth rate (assumed)
Bangalore	10%
Mumbai, Hyderabad	9%
Pune, Ahmedabad	8%
Chennai, Delhi	7%
Kolkata	6%
All other cities	8% (base rate)

Important: The dataset does not contain real future prices, and the city growth rates are assumptions made for this project. The regression target is therefore a rule-based proxy, not a measured market outcome. This is discussed in Section 4 (Limitations).

1.5 Feature sets

- **Classifier features:** all numeric features and categorical features, **excluding** Price_in_Lakhs and Price_per_SqFt. These two columns were excluded because the label is calculated from price per sq ft. Including them would let the model copy the label rule instead of learning from other features.
- **Regressor features:** all numeric features (including Price_in_Lakhs) and categorical features. The regressor needs the current price, because the target is calculated from it.

1.6 Train/test split and preprocessing

- **Split:** 80% training (200,000 rows) and 20% test (50,000 rows), using `train_test_split` with stratification on Good_Investment.
- **Imputation:** Numeric missing values are filled with the median. Categorical missing values are filled with the most frequent value. This is done inside the pipeline, so the same steps run on new data.
- **Pipeline:** All preprocessing and the model are combined into one scikit-learn Pipeline, which avoids data leakage from test data into training.

1.7 Models

Classification (Good Investment)

Model	Settings
Logistic Regression	max_iter = 2000, scaled features
Random Forest	100 trees, max depth 15, min samples per leaf 5
XGBoost	400 trees, max depth 6, learning rate 0.05, subsample 0.8, column sample 0.8

Random Forest was trained on a random sample of 60,000 training rows to keep training time reasonable. The other models used all training rows.

Regression (5-year future price)

The model learns the **growth ratio** (future price ÷ current price), and the prediction is multiplied by the current price. This approach was chosen because tree-based models cannot predict values outside the range seen in training. Predicting a ratio keeps the output sensible for any price.

Model	Settings
Fixed-rate baseline	Every property grows at 8% per year (no learning)
Linear Regression	Scaled features
Random Forest	100 trees, max depth 15, min samples per leaf 5
XGBoost	500 trees, max depth 6, learning rate 0.05, subsample 0.8, column sample 0.8

1.8 Model selection and evaluation

- **Classification:** The best model is chosen by the **F1 score** on the test set. Accuracy, F1, ROC-AUC and the confusion matrix are reported.
- **Regression:** The best model is chosen by **RMSE** on the test set. RMSE, MAE and R^2 are reported. The fixed-rate baseline is included for comparison.

Note on model selection: The best model was selected using test-set results, because the project does not have a separate validation set. The reported test scores therefore slightly overstate how well the chosen model will generalise. This is listed in Section 4.

1.9 Experiment tracking

- All training runs were logged to **MLflow** under the experiment name `real_estate_advisor`.
- Logged items: model name, metrics (accuracy, F1, ROC-AUC, RMSE, MAE, R^2) and the fitted pipeline.
- The best classifier (Random Forest) and the best regressor (Random Forest) were registered in the **MLflow Model Registry** as `real_estate_classifier` (version 1) and `real_estate_regressor` (version 1).
- The MLflow log files are stored in `mlflow_logs/`, and can be viewed with: `mlflow ui --backend-store-uri sqlite:///mlflow_logs/mlflow.db`
- Earlier runs from the experiment history (for example, runs from before the regression ratio target and the Random Forest limits were introduced) are kept in the logs, but they are not used for the final results.

1.10 Output artifacts

File	Purpose
`models/classifier.joblib`	Saved Good Investment classifier (Random Forest pipeline)
`models/regressor.joblib`	Saved 5-year price regressor (Random Forest pipeline)
`reports/metrics.json`	All metrics, confusion matrix and top features
`data/app_sample.csv`	30,000-row sample used by the Streamlit app
`cleaned_housing_data.csv`	Full cleaned dataset (250,000 rows, 25 columns)
`mlflow_logs/`	MLflow experiment logs and registry